Core Functionalities Development

EduGenie - AI Learning Assistant

1. Objective

This stage turned the architecture into working features. It covers the five learning tools, the routes that connect them to the web page, and the validation and error handling that keep the system stable. For each feature this document explains how it is made and how it helps the learner.

2. Feature Overview

Feature	File	How it is made	How it helps the learner
Concept Explanation	explanation_module.py	Local LaMini-Flan-T5 model via Transformers	Quick plain-language definition without internet cost
Question Answering	qna.py	Gemini with an instruction for detailed, example-rich answers	Deeper understanding of any question
Summarisation	summary_module.py	Gemini with overview + bullet format	Saves reading time on long text
Quiz Generation	quiz_module.py	Gemini with a 3-question MCQ format and Answer Key	Instant self-assessment
Learning Path	learning_path.py	Gemini with phased roadmap format	A study plan with stages, projects and timelines

3. Concept Explanation - Local Model

3.1 How it is made

The function load_model() stores the tokenizer and model in two module-level variables, _tokenizer and _model. If they are empty, it loads MBZUAI/LaMini-Flan-T5-783M with AutoTokenizer and AutoModelForSeq2SeqLM and prints progress messages to the console. On later calls the stored objects are returned, so the model is loaded only once.

get_concept_explanation() then (1) rejects empty input, (2) wraps the text in the instruction "Explain this concept clearly and concisely", (3) tokenises it as PyTorch tensors with truncation at 512 tokens, (4) calls model.generate() and (5) decodes the result without special tokens.

Setting	Value	Purpose
max_length (input)	512	Keeps very long inputs within the model's limit
max_new_tokens	256	Limits answer length and response time
do_sample	False	Greedy decoding gives the same answer for the same input
skip_special_tokens	True	Removes internal markers from the text

3.2 How it is useful

Students get a short explanation immediately, with no API key usage or per-request cost. Because decoding is deterministic, the result is repeatable, which helps when the feature is tested.

4. Gemini-Based Features

The four cloud features share one pattern, which keeps the code small and consistent:

- **Check:** return a friendly message if the input is empty, without calling the API.
- **Build:** create formatted_prompt = task instructions + formatting rules + the user's text (stripped of extra spaces).
- **Call:** send the prompt with generate_with_retry(client, formatted_prompt).
- **Protect:** wrap any failure in a RuntimeError that names the task, such as "Quiz generation failed: ...".

Module	Formatting rules placed in the prompt	Why these rules
qna	Clear answer, detailed explanations, practical examples	Learners understand better with worked examples
summary_module	1-2 sentence overview first, then bullet takeaways, simple language	Easy to scan; the overview gives instant context
quiz_module	3 questions, options A-D, final 'Answer Key' with brief explanations	Gives a fixed, easy-to-read quiz and teaches through explanations
learning_path	Phases (Beginner, Intermediate, Advanced), key topics, project ideas, timelines, Markdown headers and bullets	A roadmap needs order, practice and time estimates

5. Backend Routes and Routing Logic

Route	Method	What it does
/	GET	Renders index.html with response=None, prompt empty and task=concept (the first-visit state).
/generate	POST	Reads prompt and task from the form, calls the matching module, stores the answer and the model label, and renders index.html.

The TaskType enum is the contract between the page and the server. Because task is declared as TaskType in the route signature, FastAPI validates it automatically and the if/elif chain only ever sees one of five legal values. The Gemini model label is stored once in the GEMINI_LABEL constant, so it can be changed in a single place.

6. Validation and Error Handling

Layer	What is checked	What the user sees
Browser	Textarea has the required attribute	The form cannot be sent empty
Start-up	API key present and not an OAuth token	Server refuses to start with a clear message
Module	Input empty after trimming	"Please provide ..." message, no model call
Gemini call	Any exception from the API	RuntimeError naming the failed task
Local model	Any exception while loading or generating	"Error running local model: ..."

Layer	What is checked	What the user sees
Route	Any exception reaching generate_response()	"Error generating response: ..." with the label System Error

A comment in Main.py records an important fix: the final return statement sits outside the try/except block, so the page is returned for both success and error cases. This means the user always gets a rendered page back, with either the answer or an error message.

7. Example Flow: Learning Path

```
Input   : task = path, prompt = "Data Science"
Route   : TaskType.PATH -> learning_path.generate_roadmap(gemini_client, prompt)
Prompt  : "Create a comprehensive, step-by-step learning roadmap ... Phase 1 ... Phase 3 ..."
Call    : generate_with_retry(client, formatted_prompt)
Output  : Markdown-style roadmap text + model_used = Gemini label
Display : shown inside the response box of index.html
```

8. Outcome

All five tools are independent, validated and exposed through one endpoint. Their text output, together with the model label, is what the Frontend Development stage presents to the student.